CD 80

7F 45 4C 46

计算机系统 实验三

最小ELF可执行文件

分阶段破解式实验指南（学生版）

深化对ELF格式与链接的相关认知

体验古法手工工艺之美

02 00 03 00

01 00 00 00

学生信息

姓名：

学号：

班级：

实验日期：

实验目录

实验简介 实验目标与机制说明

阶段一 C程序基础分析与优化

阶段二 汇编改写与系统调用

阶段三 去除库依赖与链接优化

阶段四 手动构建ELF文件头

阶段五 结构重叠优化

实验简介

实验背景

在计算机系统中，可执行文件是程序运行的载体。ELF（Executable and Linkable Format）是Linux系统中最常用的可执行/可链接文件格式。本实验将带领你探索ELF文件的内部结构，通过一系列优化手段，将一个原本约数十K字节的可执行文件逐步压缩到尽可能最小。

实验目标

学习目标

- 深入理解ELF文件结构
- 掌握Linux系统调用机制
- 理解链接器的工作原理
- 学习汇编语言编程基础
- 体验极致优化的思维方式

个性化设置

本实验采用个性化设计，每位学生的目标返回值（即 return的数值）为其学号的末尾6位数字。

你的学号

_____

你的个性化目标值（学号末6位）

_____（请填写）

实验机制

本实验采用分阶段破解式设计：

1. 6个阶段：实验分为6个难度递增的阶段
2. 任务驱动：每个阶段包含具体任务和填空题
3. 密钥解锁：完成阶段任务后可获得解锁密钥
4. 循序渐进：必须完成前一阶段才能进入下一阶段

> 重要提示
>
> 本实验需要你具备一定的C语言基础和Linux命令行操作经验。部分阶段涉及x86汇编语言，不用担心，我们一边回顾一边探索。（本实验建议优选在32位系统进行。）

由于需要对汇编代码直接进行修改，请一定采用更有力的汇编语言编译器nasm：

- **安装**：可以通过包管理器安装，例如在 `Ubuntu` 上使用 `sudo apt install nasm`。
- **编写代码**：使用文本编辑器编写汇编代码，文件扩展名通常为 `.asm`。
- **编译代码**：使用命令 `nasm -f format filename.asm` 将汇编代码编译成目标文件。
- **链接**：使用链接器（如 `ld`）将目标文件链接成可执行文件

阶段一：C程序基础分析与优化

阶段目标

从最简单的C程序开始，了解编译器生成的可执行文件大小，学习使用基本优化手段减小文件“尺寸”。

前置知识

- C语言基础语法
- gcc编译器基本用法
- Linux基本命令（wc, echo等）

实验任务

任务1.1：编写基础C程序

创建一个名为 `tiny.c` 的文件，内容如下（将你的目标值填入）：

```
/* tiny.c */
int main( ) { return 你的目标值 ; }
```

任务1.2：编译并查验大小

执行以下命令编译程序并查看返回值：

```
$ gcc -Wall tiny.c $
./a.out ; echo $?
目标值
$ wc -c a.out
文件大小 a.out
```

问：这个可执行文件有多大？如果要减小可执行代码，从C代码层次能进一步简化吗？求助于gcc呢？

热身练习

继续下一步之前，先演练一下nasm，以下是一个手写汇编的示例及其编译过程：

```
;Example.asm：
BITS 32      ; 指定生成 32 位代码
GLOBAL main   ; 声明 main 为全局符号
SECTION .text ; 定义代码段

main:
   mov eax, 5
   mov ebx, 10
   add eax, ebx
 ret

nasm编译命令：
nasm -f elf32 Example.asm
gcc -m32 -Wall -s Example.o
```

Tip: 请运行一下该示例并查看其返回值，这对后续可能的汇编代码编写会有帮助。

任务1.3：使用strip优化

使用 -s 选项

移除可执行文件中的部分内容：

```
$ gcc -Wall -s              tiny.c
$ wc -c a.out
优化后大小                  a.out
```

任务1.4：使用编译器优化

```
$ gcc -Wall -s -O3          tiny.c
$ wc -c a.out
最终大小                    a.out
```

思考题

1. strip命令移除了什么内容？为什么可以减小文件大小？

2. 为什么-O3优化对这么简单的程序效果不明显？

3. 你认为这个C程序生成的可执行文件中，实际执行的代码占多大比例？

阶段验证记录

完成上述任务后，请记录：

项目	大小（字节）

原始文件大小	_____
strip后大小	_____
-O3优化后大小	_____

阶段一解锁密钥

将上述三个数字用连字符连接，格式：`数字1-数字2-数字3`

你的密钥：_____

下面将进入阶段二……

阶段二：汇编改写与系统调用

阶段一 | **阶段二** | 阶段三 | 阶段四 | 阶段五 | 阶段六

阶段目标

将C程序改写为汇编语言，去除C运行时库的依赖，直接使用Linux系统调用退出程序。

> **知识补充：Linux系统调用**
>
> Linux通过 `int 0x80` 中断提供系统调用接口。调用前需要设置：
>
> - `eax`：系统调用号（exit为1）
> - `ebx`：第一个参数（返回值）

实验任务

任务2.1：编写汇编程序

创建 `tiny.asm` 文件，填写缺失的值：

```
; tiny.asm
  BITS        32
  GLOBAL _start
  SECTION  .text
  _start:
                mov     eax, ?                  ;exit系统调用号
                mov     ebx,  你的目标值                ; 返回值
                int     ?               ;  触发系统调用
```

任务2.2：编译汇编程序

```
$ nasm -f elf tiny.asm
$ ld -s tiny.o
$ ./a.out ; echo $?
目标值
$ wc -c a.out
文件大小                    a.out
```

任务2.3：指令优化

分析当前程序的机器码大小，尝试使用更短的指令：

```
; 原始指令（？字节）
mov eax,1           →B8 01 00 00 00

; 优化后（？字节）
xor eax,eax         →31 C0
inc eax             →40

;  原始指令（？字节）
mov ebx, 42         →BB 2A 00 00 00

;  优化后（？字节）
mov bl, 42          →B3 2A
```

优化后的完整程序：

```
; tiny.asm
    BITS 32
    GLOBAL _start
    SECTION .text
    _start:
                    xor     eax,   eax          ; 2字节;
                    inc     eax                 ; 1字节
                    mo      bl,      你的目标值          ;  2 字节
                    int     0x80            ;  2 字节
```

阶段验证记录

项目	大小（字节）

优化前文件大小	_____
优化后文件大小	_____
节省的字节数	_____

阶段二解锁密钥

计算：优化后大小 × 4 + 节省字节数

你的密钥：_____

阶段三：去除库依赖与链接优化

阶段目标

理解ELF文件中的节（Section）概念，学习如何去除“非必要”内容，使用ld直接链接。

> 知识补充：ELF文件结构
>
> ELF文件包含两个重要的表：
>
> - 节头表（Section Header Table）：用于编译器和链接器，描述各节的位置和属性
> - 程序头表（Program Header Table）：用于程序加载器，描述如何加载程序到内存
>
> 记得吗？对于可执行文件，节头表是可选的！why?

实验任务

任务3.1：分析当前ELF结构

使用 readelf 或 objdump 查看当前可执行文件的结构与内容，比如：

```
$ readelf -a a.out
```

观察输出，会发现：我们并没有调用外部库的操作，怎么会出现相关链接操作？

gcc时，使用 -nostdlib ， 将取消链接标准库和相关启动代码，

然后你就会遇到 _start未定义报错，why?

_start 是程序运行的入口，我们可以自己定义，而且我们的C代码如此简单，完全可以让代码

在_start中运行，不需要main函数了！

任务3.2：直接对_start 编程

改写代码直接在_start中完成，成功编译，执行……

……出现段错误？！（不慌不慌……）

因为 _start 并不是一个C函数（那_start到底是什么？），无法从其返回。

请回忆，出现函数调用时，栈的内容是什么？

请在当前栈上找到argc，argv ，envp等内容。

argc，argv，envp是干什么用的？

到底如何从_star退出？

- 与大多数操作系统一样， Linux 通过系统调用为其托管的程序提供基本操作，包括打开文件、读取和写入文件句柄等，当然，还包括关闭进程。
- 用 int 0x80 指令唤醒内核
- 若要结束该进程，并返回相应的值，需要对寄存器eax赋值为1（告诉内核结束进程），对ebx赋返回值——请思考，为什么只能对这两个寄存器赋值？

请再次查看文件，与之前有什么变化？

任务3.3：使用ld直接链接

之前使用gcc链接依然会引入不必要的节，所以我们自己直接使用链接器ld：

```
ld -m elf_i386 Example.o
 //elf_i386是指生成32 位 x86 架构的 ELF 格式
```

现在 _start 修改后的内容是：

```
xor eax, eax
inc eax
mov bl,
int 0x80
```

```
$ nasm -f elf tiny.asm
$                -s tiny.o
$ wc -c a.out
文件大小                    a.out
```

任务3.4：理解大小变化

对比gcc和ld链接的结果，思考并回答：

1. gcc链接时默认添加了哪些内容？TA为什么要增加这些内容？

---

2. ld链接的文件为什么更小？

ELF文件头结构详解

ELF文件头（52字节）包含以下关键字段：

字段	偏移	大小	说明
e_ident	0	16字节	魔数和标识信息
e_type	16	2字节	文件类型（2=可执行）
e_machine	18	2字节	目标架构（3=x86）
e_version	20	4字节	版本（1）
e_e n tr y	24	4字节	程序入口地址
e_phoff	28	4字节	程序头表偏移

e_shoff	32	4字节	节头表偏移（0=无）
e_flags	36	4字节	处理器特定标志
e_ehsize	40	2字节	ELF头大小（52）
e_phentsize	42	2字节	程序头表项大小
e_phnum	44	2字节	程序头表项数量
e_shentsize	46	2字节	节头表项大小
e_shnum	48	2字节	节头表项数量（0=无）
e_shstrndx	50	2字节	节名字符串表索引

阶段验证记录

ld链接后的文件大小：_____ 字节

阶段三解锁密钥

密钥格式：`LD-字节数`

你的密钥：_____

阶段四：手动构建ELF文件头

阶段目标

使用nasm的bin输出格式，手动构建一个完整的ELF可执行文件，精确控制每个字节。

> 知识补充：bin输出格式
>
> nasm的 -fb in选 项输出纯二进制文件，不会添加任何ELF结构。我们需要自己手动构建完整的ELF头和程序头表。

程序头表结构

程序头表项（32字节）描述一个内存段：

字段	偏移	大小	说明
p_type	0	4字节	段类型（1=可加载）
p_offset	4	4字节	文件偏移
p_vaddr	8	4字节	虚拟地址
p_paddr	12	4字节	物理地址（忽略）
p_filesz	16	4字节	文件大小
p_memsz	20	4字节	内存大小
p_flags	24	4字节	权限标志（5=读+执行）

p_align	28	4字节	对齐要求

实验任务

任务4.1：编写ELF模板

创建 `tiny.asm` 文件，填写缺失的值：

```
; tiny.asm
  BITS 32
                org     0x08048000

  ehdr:                                                 ; Elf32_Ehdr
                db      0x7F, "ELF", 1, 1, 1, 0         ;   e_ident
          times 8 db      0
                dw      ?                                       ;e_type
                dw      ?                                       ;e_machiee
                dd      ?                                       ;e_version
                dd      _start                          ;   e_entry
                dd      phdr - $$                       ;   e_phoff
                dd      ?                                       ;   e_shoff
                dd      ?                                       ;   e_flags
                dw      ehdrsize                        ;   e_ehsize
                dw      phdrsize                        ;   e_phentsize
                dw      ?                                       ;   e_phnum
                dw      ?                                       ;   e_shentsize
                dw      ?                                       ;   e_shnum
                dw      ?                                       ;   e_shstrndx

  ehdrsize      equ     $ - ehdr

  phdr:                                                 ; Elf32_Phdr
                dd      ?                                       ;   p_type
                dd      ?                                       ;   p_offset
                dd      $$                              ;   p_vaddr
                dd      $$                              ;   p_paddr
                dd      filesize                        ;   p_filesz
                dd      filesize                        ;   p_memz
                dd      ?                                       ;   p_flags
                dd      0x1000                          ;   p_align

  phdrsize      equ     $ - phdr

  _start:
                mov     bl,   你的目标值
                xor     eax, eax
                inc     eax
                int     0x80

  filesize      equ     $ - $$
```

任务4.2：编译并测试

```
$ nasm -f bin -o a.out tiny.asm
$ chmod +x a.out
$ ./a.out ; echo $?
目标值
$ wc -c a.out
文件大小                a.out
```

任务4.3：分析文件结构

使用xxd查看文件内容：

```
$ xxd a.out | head -10
```

验证ELF魔数（前4字节）：**7F 45 4C 46**

思考题

1. ELF头大小 + 程序头表大小 = _____ 字节
2. 程序代码大小 = _____ 字节
3. 文件总大小是否等于上述之和？为什么？

阶段验证记录

文件总大小：_____ 字节

> 阶段四解锁密钥
>
> 计算：文件大小 × 2 - 程序代码大小
>
> 你的密钥：
> _____

阶段五：结构重叠优化

阶段目标

通过让程序头表与ELF头重叠，进一步减小文件大小。这是ELF优化的关键技巧。

> 重叠原理: ELF头的最后8字节（e_shentsize, e_shnum, e_shstrndx）与程序头表的前8字节（p_type, p_offset）在位置上重叠。如果我们能让这两个区域的内容同时满足两个结构的语义，就能节省8个字节！

重叠布局分析

```
ELF头末尾:
  e_shentsize (2 字节) = 0
  e_shnum     (2 字节) = 0
  e_shstrndx  (2 字节) = 0

程序头表开头:
 p_type      (4字节) = 1   (可加载段)
p_offset     (4字节) = 0   (文件偏移)
```

关键观察：e_phnum（值为1）可以与p_type（值为1）重叠！

实验任务

任务5.1：编写重叠版本

```
; tiny.asm
  BITS 32
                org     0x08048000

  ehdr:
                db      0x7F, "ELF" 1, 1, 1, 0,         ; e_ident
                db      0
_start:         mov     bl,     你的目标值
                xor     eax, eax
                inc     eax
                int     0x80
                dw      ?                               ; e_type
                dw      ?                               ; e_machine
                dd      ?                               ; e_version
                dd      _start                  ; e_entry
                dd      phdr - $$               ; e_phoff
                dd      ?                               ; e_shoff
                dd      ?                               ; e_flags
                dw      ehdrsize                ; e_ehsize
                dw      phdrsize                ; e_phentsize
  phdr:         dd      ?                       ; e_phnum ; p_type
                                                ;e_shentsize
                dd      ?                       ;e_shnum        ;p_offset
                                                ;e_shstrndx
  ehdrsize      equ     $ - ehdr
                dd      $$                                      ; p_vaddr
                dd      $$                                      ; p_paddr
                dd      filesize                                ; p_filesz
                dd      filesize                                ; p_memsz
                dd      ?                                               ; p_flags
                dd      0x1000                                  ; p_align
  phdrsize      equ     $ - phdr

  filesize      equ     $-$$
```

任务5.2：编译并验证

```
$ nasm -f bin -o a.out tiny.asm
$ chmod +x a.out
$ ./a.out ; echo $?
目标值
$ wc -c a.out
文件大小                a.out
```

任务5.3：进一步优化

改变加载地址，让更多字段可以复用：

```
org 0x00010000;     更小的地址使 e_entry 值更小
```

将p_flags从5改为4（这也行？？）：

```
dd 4   ; p_flags    （读权限）
```

阶段验证记录

最终文件大小：_____ 字节

阶段五解锁密钥

密钥格式：`ELF- 字节数`

你的密钥：_____

思考题

1、为什么加载地址可以被改到如此“低”的数字？

2、p_flags字段为什么能这样修改？

阶段六：终极极限挑战

阶段目标

尝试达到ELF可执行文件的绝对最小值！利用Linux加载器的容错性，移除文件末尾的零字节。

终极挑战

这个阶段需要你完全理解ELF格式和Linux加载器的行为。最终的最小字节数的可执行程序中，大部分字段的值都违反了ELF标准，但Linux仍然会加载并执行它！为什么？

最后的优化技巧

技巧1：文件截断

Linux加载器会为零字节字段自动补零。我们可以移除文件末尾的零字节！

技 巧 2： p_ filesz与 p_ memsz的 关 系

当内存段只读时，p_memsz不能大于p_filesz（因为无法零初始化）。解决方案：让p_filesz = p_memsz，即使它们都比实际文件大！

技巧3：e_phoff的妙用

e_phoff（程序头表偏移）的值4，恰好可以作为p_flags的值（读权限）！

终极代码框架

```
; tiny.asm -      可能最小字节数终极版本
  BITS 32
                  org     0x00010000

                  db      0x7F, "ELF"             ; e_ident (4字节)
                  dd      ?                                       ; p_type
                  dd      ?                                       ; p_offset
                  dd      $$                              ; p_vaddr
                  dw      ?                       ; e_type        ; p_paddr
的高2字节
                  dw      ?                       ; e_machine     ; p_paddr
的低2字节
                  dd      _start                  ; e_version     ; p_filesz
                  dd      _start                  ; e_entry       ; p_memsz
                  dd      ?                               ; e_phoff       ; p_flags
  _start:
                      mov     bl,   你的目标值                    ; e_shoff     ;   p_align

                  xor     eax,eax
                  inc     eax                     ; e_flags
                  int     0x80
                  db      ?
                  dw      0x34                    ; e_ehsize
                  dw      0x20                    ; e_phentsize
                  dw      ?                        ; e_phnum
                      这里还有什么？               ;e_shentsize ;e_shnum ; e_shstrndx

  filesize        equ     $ - $$
```

实验任务

任务6.1：完成终极代码

根据提示填写所有空白处，特别注意：

- e_phoff = 4，同时作为p_flags = 4（读权限）
- p_filesz和p_memsz都等于e_entry（入口地址）
- 最后的e_phnum使用db（1字节）而非dw（2字节）

任务6.2：编译验证

```
$ nasm -f bin -o a.out tiny.asm
$ chmod +x a.out
$ ./a.out ; echo $?
目标值
$ wc -c a.out
                          a.out
```

任务6.3：预期能够获得最小可执行文件大小是45字节，其构成如下。你的呢？

部分	字节数	说明
e_ident	4	魔数 7F 45 4C 46
程序头表	28	前7个字段
程序代码	7	mov bl, xx; xor eax,eax; inc eax; int 0x80
ELF头剩余	5	e_ehsize, e_phentsize, e_phnum
填充	1	db 0
总计	45	绝对最小值！

终极思考题

1. 为什么e_phnum可以使用1字节（db）而不是2字节（dw）？

2. p_filesz和p_memsz都比实际文件大小大很多，为什么程序还能正常运行？

3. 这个最小字节量的程序有多少字段违反了ELF标准？

4. 从几十K字节优化到现在，压缩比是多少？

实验完成证书密钥

恭喜你完成了全部6个阶段！

最终密钥参考值：`TINY-ELF-45`

你的密钥是多少？

实验提交

阶段密钥汇总

请填写你在各阶段获得的密钥：

阶段	密钥
阶段一	_____
阶段二	_____
阶段三	_____
阶段四	_____
阶段五	_____
阶段六	_____

实验总结

请简述你通过本实验学到了什么（至少100字）：